Comprehensive code/logic/data quality audit

QuillMD — pilot readiness

Generated: 2026-05-14T03:04:26Z
Lead: Claude (Opus 4.7 1M)
Trigger: Pre-pilot-doctor onboarding gate
Method: 4-phase audit (inventory → triage → deep-dive → deliverables)

P0	**0**	Pilot-blocking, active harm. None after ground-truth verification.
P1	**18**	Real bug or significant gap; high pilot impact.
P2	**35**	Hardening / robustness item; no immediate user impact.
P3	**11**	Cosmetic / nice-to-have / backlog.
Total	**64**	Below the 200-finding triage-narrowing threshold.

Pilot-readiness verdict: GREEN with documented P1 follow-ups

The audit's deep dive on production data shows the system is operationally healthy. 201 visits across 30 days. 9 containers all healthy. Stuck-visit watchdog catches its own incidents. DLQ landed (1 wait / 0 failed). audit_logs at 6,967 rows with multi-tenant scoping enforced. No P0 means no active user harm and no pilot-blocker.

Contents

Executive summary ... A — Section A

Methodology ... B — Section B

Detailed findings (P0 + P1) ... C — Section C

FIND-P1-001 to FIND-P1-018 ... (18 deep-dive entries)

P2 + P3 grouped summaries ... D — Section D

Domain rollups ... E — Section E

Appendix ... F — Section F

A — Executive summary

Pilot-readiness verdict: GREEN with documented P1 follow-ups. The audit found 0 P0, 18 P1, 35 P2, 11 P3 across 12 domains. Three sub-agent P0_candidates were downgraded to P1 after production-data ground truth — the verification discipline is intentional. P0 is reserved for active user harm; the audit's job is not to manufacture severity.

Theme clusters in the P1 tier:

1. Audit-emission gaps on server-action paths — 3 findings in `actions/soap.ts` (`signVisitAction`, `saveSoapDraftAction`, `reassignSpeakerAction`). Route-path equivalents emit (production has 7 `VISIT_SIGNED` rows); action path is a duplicate surface that skips audit. Single-doctor pilot can avoid this; should close before multi-doctor pilot.

2. Retry/jitter cluster — 4 findings in `packages/ai` + L1/L1b. Deepgram/Whisper batch calls have zero inline retry; L1's retry uses `setTimeout` without jitter. Single-doctor pilot unlikely to hit thunder-herd; multi-doctor pilot will.

3. Pipeline output validation gaps — 3 findings in L2/L4/L6. L4 SOAP and L6 polish write model output without Zod validation; L2's fallback path skips its own schema check. Mitigations work today; a model regression or prompt drift would leak malformed content to the doctor's screen.

4. Worker error-handling depth — 4 findings in `worker.ts`. `loadResumeState` P2025 race on visit deletion; nested `.catch()` can leave visits in non-terminal status if both writes fail; `shouldFailNow` categorize-error false-positive bypasses retries.

Top 5 most-critical findings:

P1 **FIND-P1-001** — signVisitAction HIPAA audit-trail gap on the action path.

P1 **FIND-P1-015** — L4 SOAP generation: model output not Zod-validated; corrupt JSON could flow to L5/L6 unchecked.

P1 **FIND-P1-014** — Worker FAILED-state update has nested .catch() inside outer catch — visit stuck non-terminal if both writes fail.

P1 **FIND-P1-018** — Pipeline layers L2-L6 have NO sibling unit tests; critical-path test gap.

P1 **FIND-P1-007** — Whisper fallback has NO retry — transient OpenAI 5xx → hard FAILED state with no automatic recovery.

Already-audited domains (skipped):

Security (PRs #240-#244), cross-input-mode (#237), keyterm scoping + ISMP safety floor (#246-#250, Issues #202+#206), rate-limit (#238), email deliverability (#245). The Phase 1 inventory does not re-find any of these.

B — Methodology

Four-phase audit pipeline:

Phase 1. Surface scan across 12 domains (A backend/worker, B database, C pipeline, D API, E frontend, F code-quality, G performance, H reliability, I data-integrity, J tests, K a11y, L operational). Output: phase1-inventory.md with 66 candidate findings.

Phase 2. Triage against production-data ground truth. 3 sub-agent-claimed P0s downgraded to P1 after verification (signVisitAction → route-path emits in prod; saveSoapDraftAction → DoctorEditPattern captures; 07b dual-write → BullMQ retry mitigates).

Phase 3. Deep-dive on every P0 + P1: file:line, evidence, recommended fix, effort, dependencies.

Phase 4. Deliverables: Markdown report (this file's source-of-truth), PDF, CSV findings export.

Verification discipline:

Three sub-agent P0_candidates were re-checked against production data:

Sub-agent claim	Production check	Verdict
`signVisitAction:165` has no `logAudit`	grep `VISIT_SIGNED` → emitted at route.ts:269; 7 prod rows	P1 action-path partial gap
`saveSoapDraftAction:84` no audit	grep `EDITED_NOTE` in source → 0 emissions; captured via DoctorEditPattern	P1 visibility gap, not data-loss
`07b-cpt-coding` dual-write not in $transaction	Read code; confirmed; BullMQ retry + watchdog mitigate	P1 real, but mitigated

This downgrade discipline is intentional. P0 is reserved for active user harm; the audit's job is not to manufacture severity.

C — Detailed findings (P0 + P1)

No P0 findings after triage. Three sub-agent P0_candidates were verified and downgraded to P1 (see §B). The pilot-blocking active-harm tier is empty.

P1 FIND-P1-001 — signVisitAction HIPAA audit-trail gap (action path)

Severity	P1	**Domain**	Reliability / HIPAA audit completeness
Location	`apps/transcribe-md/src/actions/soap.ts:130`	**Effort**	Small
Dependencies	None		

Description. The server-action signVisitAction mutates status=SIGNED + signedSoap + signedAt — the legal signing event. It does NOT call logAudit. The parallel route /api/visits/[id]/sign/route.ts:269 DOES emit VISIT_SIGNED, and production has 7 such rows generated through it. Two surfaces are duplicated; doctors landing on the action path skip the audit.

Evidence:

```
await prisma.transcribeVisit.update({ where: { id: visitId },
  data: { status: TranscribeVisitStatus.SIGNED, signedSoap: visit.soapFinal, signedAt: new Date() } });
void captureEditPattern({ visitId, doctorId: session.user.id });
// no logAudit adjacent
```

Recommended fix. Add logAudit({action:'VISIT_SIGNED', targetType:'TranscribeVisit', targetId:visitId, userId, practiceId, metadata:{source:'server_action'}}) immediately after the update. Alternatively, extract sign logic into a shared helper called by both surfaces.

P1 FIND-P1-002 — saveSoapDraftAction SOAP-edit audit visibility gap

Severity	P1	**Domain**	Reliability / HIPAA audit completeness
Location	`apps/transcribe-md/src/actions/soap.ts:42`	**Effort**	Small
Dependencies	None		

Description. Saves SOAP edits via optimistic-concurrency. Edit pattern is captured in DoctorEditPattern table; audit_logs has no row. audit_logs.action enum lists EDITED_NOTE (36 prod rows, all legacy) and CREATED_CLINICAL_NOTE (46 prod rows, legacy) — but neither is emitted from current source. Visibility gap, not data-loss.

Evidence:

```
grep -rn 'EDITED_NOTE' apps/transcribe-md/src → empty
grep -rn 'CREATED_CLINICAL_NOTE' apps/transcribe-md/src → empty
psql: 36 EDITED_NOTE rows, all legacy
```

Recommended fix. Decide policy: (a) emit EDITED_NOTE on every save (high cardinality), or (b) add a SOAP_DRAFT_SAVED debounced action. Either way, retire the dead enum values in a cleanup PR.

P1 FIND-P1-003 — reassignSpeakerAction no audit emission

Severity	P1	**Domain**	Reliability / HIPAA audit completeness
Location	`apps/transcribe-md/src/actions/soap.ts:197`	**Effort**	Small
Dependencies	None		

Description. Doctor reassigns speaker role (L2 override) — mutates speakerMap, affects clinical attribution in the final note. No logAudit emission. Who reassigned what/when requires DB row state at point in time.

Evidence:

```
(No grep for SPEAKER_REASSIGNED enum — value doesn't exist; gap is straightforward)
```

Recommended fix. Add SPEAKER_ROLE_REASSIGNED enum value (migration: ALTER TYPE 'AuditAction' ADD VALUE) and emit with {previousRole, newRole, speakerId} metadata.

P1 FIND-P1-004 — L7b CPT-coding non-transactional dual write

Severity	P1	**Domain**	Data integrity
Location	`apps/transcribe-md/src/worker/layers/07b-cpt-coding.ts:219,229`	**Effort**	Small
Dependencies	None		

Description. Two sequential prisma.update calls (needsReview+reviewReason at L219, cptCodes at L229) NOT in $transaction. Worker crash between leaves visit flagged for review with no codes to verify. BullMQ retry + stuck-visit watchdog mitigate the user-visible window.

Evidence:

```
if (lowConfProcedures.length > 0) {
  await prisma.transcribeVisit.update({ data: { needsReview: true, reviewReason: ... } });
}
await prisma.transcribeVisit.update({ data: { cptCodes: filtered as unknown as object } });
```

Recommended fix. Collapse into one update with conditional spread: { cptCodes: filtered, ...reviewFields }. Atomic single-statement write, no $transaction overhead.

P1 FIND-P1-005 — actions/soap.ts:177 fire-and-forget captureEditPattern

Severity	P1	**Domain**	Async correctness
Location	`apps/transcribe-md/src/actions/soap.ts:177`	**Effort**	Small
Dependencies	None		

Description. void captureEditPattern(...) after signVisitAction's primary update. Comment says callee 'swallows every error internally' — relies on the callee's contract. If a future refactor leaks an error, rejection silently disappears and action returns success while edit-pattern is dropped.

Evidence:

```
void captureEditPattern({ visitId, doctorId: session.user.id });
// no .catch wrapper
```

Recommended fix. captureEditPattern(...).catch((err) => log.warn('actions:signVisit', 'captureEditPattern failed', { err })). The structured log gives Sentry/loki a row even if callee's internal try/catch breaks.

P1 FIND-P1-006 — Deepgram batch has no inline retry

Severity	P1	**Domain**	Reliability
Location	`packages/ai/src/deepgram-batch.ts:237`	**Effort**	Medium
Dependencies	Resolves with FIND-P1-007, P1-008, P1-009 as a single retry-helper PR		

Description. deepgramBatchTranscribe makes a single fetch with no retry. L1 layer wraps with one network retry, but non-L1 callers (eval harness, regression baseline) get zero retries on transient 5xx. A Deepgram region wobble during eval produces hard failures with no recovery.

Evidence:

```
// packages/ai/src/deepgram-batch.ts:237 — single fetch, no try/retry loop
```

Recommended fix. Add retry block inside the function: 2 attempts default, exp backoff + jitter on transient errors only. Make configurable via options so the L1 caller can pass {retries:0} to avoid double-retry.

P1 FIND-P1-007 — Whisper batch has no retry

Severity	P1	**Domain**	Reliability
Location	`packages/ai/src/whisper-batch.ts:189`	**Effort**	Small
Dependencies	Same retry helper as FIND-P1-006		

Description. Whisper is the fallback when Deepgram fails. Fallback itself has no retry — single-shot. Transient OpenAI 5xx during the rare Deepgram-failed-fallback path produces hard FAILED with no automatic recovery. Mitigation: BullMQ retry kicks the whole pipeline from L1 (expensive).

Evidence:

```
// packages/ai/src/whisper-batch.ts:189 — single fetch, no retry
```

Recommended fix. Apply the retry helper from FIND-P1-006. Default 2 attempts with exp backoff + jitter.

P1 FIND-P1-008 — L1 retry backoff has no jitter (thunder-herd surface)

Severity	P1	**Domain**	Reliability
Location	`apps/transcribe-md/src/worker/layers/01-transcribe.ts:170`	**Effort**	Small
Dependencies	1-line change		

Description. L1's retry uses await new Promise(r => setTimeout(r, BACKOFF_MS)) — no jitter. With worker concurrency=2 + future scale-out, coordinated retries on a Deepgram outage thunder-herd on recovery.

Evidence:

```
await new Promise((r) => setTimeout(r, BACKOFF_MS));
```

Recommended fix. Add jitter: const jitterMs = Math.floor(Math.random() * BACKOFF_MS); await new Promise(r => setTimeout(r, BACKOFF_MS + jitterMs));

P1 FIND-P1-009 — L1b retry backoff has no jitter (duplicate of P1-008)

Severity	P1	**Domain**	Reliability
Location	`apps/transcribe-md/src/worker/layers/01b-transcribe-dictation.ts:135`	**Effort**	Small
Dependencies	Ship with FIND-P1-008		

Description. Same defect as P1-008 in the dictation-mode parallel codepath. The duplication itself is the broader CODE_PATTERN P3 below.

Evidence:

```
// 01b-transcribe-dictation.ts:135 — same setTimeout(r, BACKOFF_MS) without jitter
```

Recommended fix. Apply jitter fix from P1-008. Better long-term: factor shared retry block into a helper.

P1 FIND-P1-010 — Anthropic JSON.parse(unfenced) on final attempt throws

Severity	P1	**Domain**	Reliability
Location	`packages/ai/src/anthropic.ts:292`	**Effort**	Small
Dependencies	None		

Description. When Anthropic tool-use response falls through to text-mode (model regression or prompt drift), code runs JSON.parse(unfenced) after stripping markdown fences. Retry-context loop catches malformed parse; if FINAL retry also returns malformed text, JSON.parse throws — caught as generic error, FAILs the visit with generic 'Processing failed' (not the doctor-actionable PipelineRejectError humanReason).

Evidence:

```
// packages/ai/src/anthropic.ts:292
return JSON.parse(unfenced);  // final-attempt path — no try/catch
```

Recommended fix. Wrap final-attempt JSON.parse in try/catch; surface as typed ModelOutputMalformedError. Worker's catch block can recognize it and surface better reviewReason.

P1 FIND-P1-011 — loadResumeState P2025 race on deleted visit

Severity	P1	**Domain**	Worker / data integrity
Location	`apps/transcribe-md/src/worker.ts:234-252`	**Effort**	Small
Dependencies	None		

Description. loadResumeState returns all-nulls fallback for missing row with comment 'next DB write will error, which is fine.' But L1/L2 updates run BEFORE re-checking row existence; visit deleted during retry causes unhandled P2025. P2025 surfaces as generic 'Processing failed during init' — not actionable.

Evidence:

```
// worker.ts:234-252
if (!row) {
  // 'next DB write will error, which is fine'
  return { /* all nulls */ };
}
```

Recommended fix. Throw a typed VisitNotFoundError when row is missing on retry; worker outer catch handles specifically (deletes the BullMQ job instead of updating non-existent row).

P1 FIND-P1-012 — confidenceScores Prisma JSON silent coerce

Severity	P1	**Domain**	Worker / data integrity
Location	`apps/transcribe-md/src/worker.ts:944`	**Effort**	Small
Dependencies	None		

Description. confidenceScores written as plain object cast to Prisma Json type. If a future layer adds a non-serializable value (Map, Set, function, Date that doesn't toISOString), Prisma silently coerces or throws opaque 'Inconsistent column data'.

Evidence:

```
// worker.ts:944
data: { confidenceScores: confidenceScores as unknown as object }
```

Recommended fix. JSON.parse(JSON.stringify(confidenceScores)) before write to fail fast on non-serializable shapes. Or: define Zod schema and parse at write site.

P1 FIND-P1-013 — shouldFailNow categorize-error false-positive bypasses retries

Severity	P1	**Domain**	Worker / reliability
Location	`apps/transcribe-md/src/worker.ts:984-1000`	**Effort**	Medium
Dependencies	None		

Description. shouldFailNow returns true when categorizeError === 'permanent', short-circuiting BullMQ retries. categorizeError source-of-truth not inspected. False-positive (transient classed as permanent) bypasses retry safety net and lands in DLQ on first attempt.

Evidence:

```
// worker.ts:984-1000
if (category === 'permanent') return true;  // skip retries
```

Recommended fix. Add unit-test suite for categorizeError asserting transient/permanent classification. Audit call sites feeding into it. Conservative 'unknown → transient' default; require 'permanent' to be explicit.

P1 FIND-P1-014 — Nested .catch() inside outer catch swallows FAILED-write failure

Severity	P1	**Domain**	Worker / reliability
Location	`apps/transcribe-md/src/worker.ts:1008-1027`	**Effort**	Small
Dependencies	None		

Description. FAILED-state persistence wrapped in .catch(() => {...}) to prevent re-throw. If BOTH original job AND FAILED-state persistence throw (e.g., DB outage), visit stays in non-terminal status (POLISHING/AUTOMATING) until 15-min stuck-visit-watchdog.

Evidence:

```
// worker.ts:1008-1027 — nested .catch swallows the FAILED-write failure silently
```

Recommended fix. Convert nested .catch to log + Sentry capture so operators see the double-failure. Then re-throw so BullMQ marks the job failed even if visit row stays mid-state. Watchdog rescues.

P1 FIND-P1-015 — L4 SOAP generation: output not Zod-validated

Severity	P1	**Domain**	AI pipeline
Location	`apps/transcribe-md/src/worker/layers/04-generate-soap.ts:214`	**Effort**	Medium
Dependencies	None		

Description. L4 calls modelClient.completeText({...}) — text variant — and writes returned text directly to soapDraft. No Zod parse, no S/O/A/P structural check, no minimum-length. If model returns empty or non-SOAP content, persisted unchecked; L5 critique gets empty input.

Evidence:

```
const { text } = await modelClient.completeText({...});
await prisma.transcribeVisit.update({ data: { soapDraft: text } });
// no structural validation between
```

Recommended fix. Add post-call check: if (!text || text.length < 200 || !/##?\s*Subjective/i.test(text)) throw new ModelOutputMalformedError(...). Throw triggers BullMQ retry on transient model issues.

P1 FIND-P1-016 — L6 polish: no structural validation on soapFinal write

Severity	P1	**Domain**	AI pipeline
Location	`apps/transcribe-md/src/worker/layers/06-polish.ts:74-89`	**Effort**	Medium
Dependencies	Ship together with FIND-P1-015		

Description. Same shape as L4 gap, in L6 polish. soapFinal (the doctor-facing output) written without structural validation. If polish strips S/O/A/P scaffolding by accident, visit reaches READY with malformed final note.

Evidence:

```
const { text } = await modelClient.completeText({...});
await prisma.transcribeVisit.update({ data: { soapFinal: text } });
```

Recommended fix. Same shape as FIND-P1-015. Reuse the structural-validation helper.

P1 FIND-P1-017 — L2 speaker-role fallback skips schema validation

Severity	P1	**Domain**	AI pipeline
Location	`apps/transcribe-md/src/worker/layers/02-speaker-roles.ts:100-115`	**Effort**	Small
Dependencies	None		

Description. L2 fallback (when AI call fails or returns no samples) writes default {0: doctor 0.5} speakerMap. AI-success branch runs through Zod parse; fallback skips it. If fallback's hardcoded shape ever drifts from schema, fallback succeeds at write but L3+ choke on malformed shape.

Evidence:

```
// 02-speaker-roles.ts:100-115 — fallback branch writes default map directly, no schema parse
```

Recommended fix. Run fallback through same Zod parse: const validated = SpeakerMapSchema.parse(fallback); await prisma.transcribeVisit.update(...). Schema drift surfaces in dev/CI not production.

P1 FIND-P1-018 — Pipeline layers L2-L6 have NO sibling unit tests

Severity	P1	**Domain**	Test coverage
Location	`apps/transcribe-md/src/worker/layers/{02-speaker-roles,03-extract-entities,04-generate-soap,05-self-critique,06-polish}.ts`	**Effort**	Large
Dependencies	None		

Description. Only L1 (low-conf-l1.test.ts), L7a (fallback), L7c (rx-drafting peds+validator) have tests. L2-L6 have ZERO sibling unit tests. The entire transformer/critique/polish chain depends on integration coverage (PR-3 regression baseline) — L2-L6 transformations not exercised by any deterministic test. Any L4/L5/L6 prompt or schema change ships with no automated guard.

Evidence:

```
ls apps/transcribe-md/src/worker/layers/__tests__/ →
  07a-icd-coding.fallback.test.ts
  07c-rx-drafting.peds.test.ts
  07c-rx-drafting.validator.test.ts
  audio-confidence.test.ts
  guards.test.ts
  low-conf-l1.test.ts
  rx-sig-parser.test.ts
```

Recommended fix. Add sibling test per layer in __tests__/. Mock modelClient.complete* to return canned responses. Assert: layer's contract (input → expected DB write shape), schema-validation branch (malformed → expected error type), fallback branch where applicable. Reuse mocking pattern from low-conf-l1.test.ts.

D — P2 + P3 grouped summaries

P2 + P3 findings (46 total) are grouped by theme rather than individually deep-dived; numbering each would inflate the document without adding triage value at this scale. The full source-of-truth list is in the Markdown report's §D.

P2 Hardening / robustness (25 themes; 35 findings)

Theme	What	Sev
Type safety: (prisma as any) casts	9 sites across admin/audit/usage data layers	P2
Type safety: session/auth as any casts	7 (session?.user as any).fieldName accesses	P2
Type safety: resume-state casts in worker	5 casts without Zod re-parse	P2
Type safety: model-response casts	3 sites (whisper-batch, anthropic, factWithEvidence)	P2
Async: fire-and-forget patterns	7 sites where errors silently dropped	P2
Worker error handling	6 P2 worker error-path findings	P2
Code patterns	5 maintainability findings (humanReason multi-cast, console.error mix, switch-vs-if, literal columns, Resend casts)	P2
Pipeline output validation depth	2 narrower-than-P1 (liftStringToFact top-level only, verify L5 modelClient enforces Zod)	P2
DB: multi-tenancy enforcement reliance	7 models depend on app-code-level practiceId join	P2
DB: AccountDeletion purge TODO	Documented TODO in schema.prisma:702-731	P2
DB: Migration irreversibility	20260412234252_add_multi_tenancy ALTER COLUMN NOT NULL on 5 tables	P2
Postgres slow-query logging disabled	log_min_duration_statement = -1	P2
Postgres pg_stat_statements not installed	No per-query mean-time telemetry	P2
Worker concurrency hardcoded	concurrency=2 — could safely scale to 4-6	P2
Frontend missing loading.tsx	patients, visits, visits/new (3 routes)	P2
Frontend missing error.tsx	patients, admin/*, signup, forgot-password (5+ routes)	P2
Frontend admin .then() without .catch	5+ admin client-side fetch sites	P2
Frontend loading-state inconsistency	34 empty-state vs 8 explicit loading-state	P2
A11y limited aria-live coverage	4 components total; visit-record + patient-picker not in list	P2
Tests: vitest coverage not configured	No c8/v8/istanbul; thresholds unenforced	P2
Tests: 72 tests vs 185 source files	~39% colocation; no critical-path matrix	P2
Tests: transcribe-md vitest NOT in CI	apps/meddoc-pro test:all is decommissioned echo (PR-4 closure §F.4)	P2
Ops: Sentry verification needed	Only 1 explicit captureException; verify worker captureWorkerException coverage	P2
Ops: postgres backup sidecar unhealthy	meddoc-pro-postgres-backup-1 Up 9 days unhealthy	P2
Ops: no heap monitoring in worker	No process.memoryUsage() polling	P2

P3 Cosmetic / backlog (11 themes; 11 findings)

Theme	What	Sev
ASYNC: setInterval void tick() sync-throw	worker.ts:1345 — defensive fix only	P3

CODE_PATTERN: L1/L1b duplication	Massive copy/paste between transcribe-dictation and transcribe	P3
DB: migration rollback artifacts	Prisma forward-only by design; PITR backup is recovery story	P3
DB: ENUM ADD VALUE pattern	Can't be wrapped in transaction; partial-deploy theoretical	P3
PERF: no bundle-analyze hygiene	Client-bundle drift untracked	P3
PERF: Docker build cache bloat	156.9 GB on host; 125.6 GB reclaimable	P3
FRONTEND: .then w/o .catch on visits rx-tab	One specific click handler — silent failure rare	P3
FRONTEND: .then w/o .catch on command palette	Recent items silently miss on rare error	P3
A11Y: no screen-reader flow audit	axe-core catches structural; semantic flow untested	P3
OPS: no alert rules	Queue depth / DLQ size / pg connections / health 5xx untracked	P3
OPS: production-recovery runbook	Oral knowledge with Kasra; documentation hygiene	P3

E — Domain rollups

Domain	P0	P1	P2	P3	Total
Backend / Worker	0	5	6	1	12
Database	0	0	6	2	8
AI Pipeline	0	3	2	0	5
Frontend / UX	0	0	4	2	6
Code quality (patterns)	0	0	16	1	17
Reliability	0	4	3	0	7
Data integrity	0	1	3	0	4
Test coverage	0	1	3	0	4
Accessibility	0	0	1	1	2
Operational	0	0	3	2	5
Audit emission (action paths)	0	3	0	0	3
Total	**0**	**18**	**35**	**11**	**64**

F — Appendix

Sources

Phase 1 inventory: docs/qa-reports/comprehensive-audit-20260514T030426Z-phase1-inventory.md (66 candidates pre-triage). code-quality-auditor sub-agent invoked once with rules library at /opt/meddoc-pro/.claude/agents/code-quality-auditor/rules.md (839 lines, 31 rules); returned 51 candidate findings for domain F. Production-data ground truth: audit_logs distribution (6,967 rows), transcribe_visits distribution (201 in 30-day window), BullMQ + DLQ state (1 wait / 0 failed), container health (9 containers).

Methodology limits

Skipped (already audited): security (#240-#244), cross-input-mode (#237), keyterm scoping + ISMP (#246-#250), rate-limit (#238), email deliverability (#245). Out of scope: semantic-flow accessibility audit (no manual screen-reader walk-through), penetration testing, performance load testing. Read-only: no fix code written.

Verification discipline

Three sub-agent P0_candidates were downgraded to P1 after production-data ground truth. The audit's job is not to manufacture severity; mitigated candidates have their severity reflect that. Downgrade history documented in §B.

Companion artifacts

Markdown source-of-truth: comprehensive-audit-20260514T030426Z.md (this PDF was generated from that file). CSV findings export: comprehensive-audit-20260514T030426Z-findings.csv (one row per finding for spreadsheet triage).

End of audit. Pilot-readiness verdict: GREEN with 18 documented P1 follow-ups. Subsequent PRs will address P1s one-by-one based on Kasra's prioritization.